TDA596 Lab 1: HTTP Server and Pub-Sub Broker

Task	Points
Concurrent HTTP file server in Go	10
Pub-sub broker in the same server	up to +7
Deploy to the cloud (AWS)	up to +3
Containers (Docker / LXC / Vagrant)	bonus

Web frameworks (Gin, Martini, …) are not allowed.

Task 1: HTTP server

Build and run

- Written in Go; compiles and runs with no errors or warnings.
- Binary is named `http_server`; first argument is the port: `./http_server <port>`.
- No hard-coded port; no assumptions about server or client IP addresses.

Concurrency

- One goroutine per client request.
- At most **10** concurrent handlers; extra requests **wait** for a free slot.

Methods and files

- `GET` serves a file; `POST` stores the request body so a later `GET` returns it.
- Supported types: (extension → content-type)
  - `.html` → `text/html`
  - `.txt` → `text/plain`
  - `.gif` → `image/gif`
  - `.jpeg` → `image/jpeg`
  - `.jpg` → `image/jpeg`
  - `.css` → `text/css`

Errors (all responses must be well-formed)

Case	Status
Unsupported extension, malformed request or headers, any other error	`400`
File not found	`404`
Valid method other than GET / POST	`501`

Atomic updates: concurrent `POST`s to the same file while others read it must be atomic. A `GET` always returns one complete version, never a partial or mixed one.

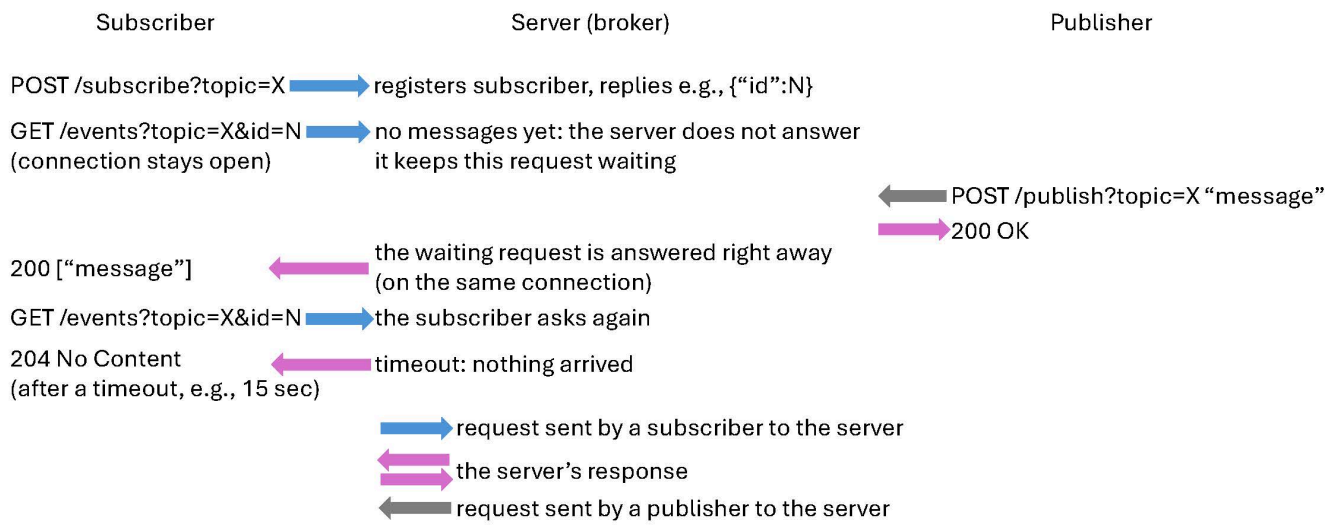
Libraries: use `net` for networking (e.g. `net.Listen("tcp", addr)`). `net/http` may be used **only** to parse requests; `http.ListenAndServe`, `http.Listen` and `http.Serve` are forbidden.

Task 2: Pub-sub broker

Publishers send messages on a topic to the broker, which forwards each message to every current subscriber of that topic. The broker never contacts subscribers itself: a message is delivered as the response to the subscriber's pending `GET /events` request. The broker runs in the same `http_server` binary and on the same port as the file server, and follows Part 1's rules.

Endpoint	Action
`POST /subscribe?topic=X`	Register a subscriber → `201` with a unique id, e.g. `{"id": N}` (unique across topics)
`POST /unsubscribe?topic=X&id=N`	Remove subscriber N
`POST /publish?topic=X`	Send the request body to every current subscriber of X
`GET /events?topic=X&id=N`	Deliver N's messages via one of the options below

`/events` delivery options:



Errors: missing topic or invalid id → `400`; unknown subscriber → `404`; wrong method → `501`.

Concurrency

- Protect shared broker state (a single mutex is fine).
- Each message gets a sequence number; subscribers receive the messages in that order.
- Subscribers only receive messages published while subscribed.
- Each subscriber has a bounded queue (e.g. a buffered channel).
  - Publishing never blocks; skip a subscriber whose queue is full.
- Waiting long polls may be excluded from the 10-goroutine limit.